

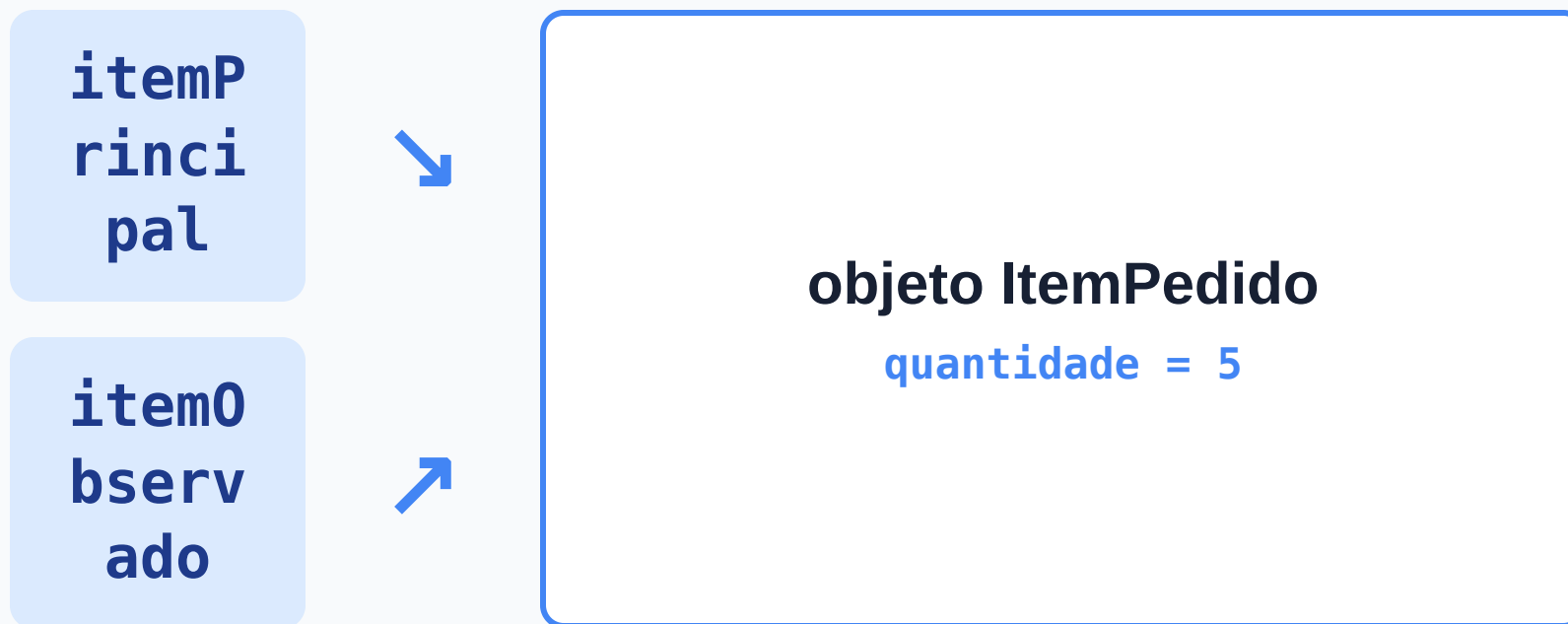
Aula 04 — Protegendo o estado dos objetos

Se diferentes partes do programa acessam o mesmo objeto, quem deve controlar como seu estado pode mudar?

O que vamos construir hoje

- diagnosticar o problema do estado exposto;
- compreender `private` como fronteira;
- controlar mudanças por comportamentos;
- distinguir encapsulamento de getters e setters automáticos.

A descoberta da Aula 03



Duas referências podem permitir acesso ao mesmo objeto.

UMA CONSEQUÊNCIA

O acesso também permite alterar

Se dois trechos chegam ao mesmo objeto, ambos podem tentar modificar seu estado.

O que acontece quando uma dessas mudanças não faz sentido para o problema?

O CENÁRIO

Duas variáveis, um objeto

```
ItemPedido itemPrincipal = new ItemPedido();  
itemPrincipal.descricao = "Teclado";  
itemPrincipal.precoUnitario = 150.0;  
itemPrincipal.quantidade = 5;  
  
ItemPedido itemObservado = itemPrincipal;
```

UMA ALTERAÇÃO

Outro trecho decide a quantidade

```
itemObservado.quantidade = -3;
```

O código ainda não foi executado.

ATIVIDADE

ANTES DE EXECUTAR

Preveja e justifique

1. Qual quantidade será observada por `itemPrincipal` ?
2. Qual será o resultado de `itemPrincipal.calcularSubtotal()` ?
3. Quantos objetos existem?

Registre primeiro. Depois compare sua justificativa com a de um colega.

A OBSERVAÇÃO

O estado compartilhado mudou

Quantidade observada

```
itemPrincipal.quantidade
```

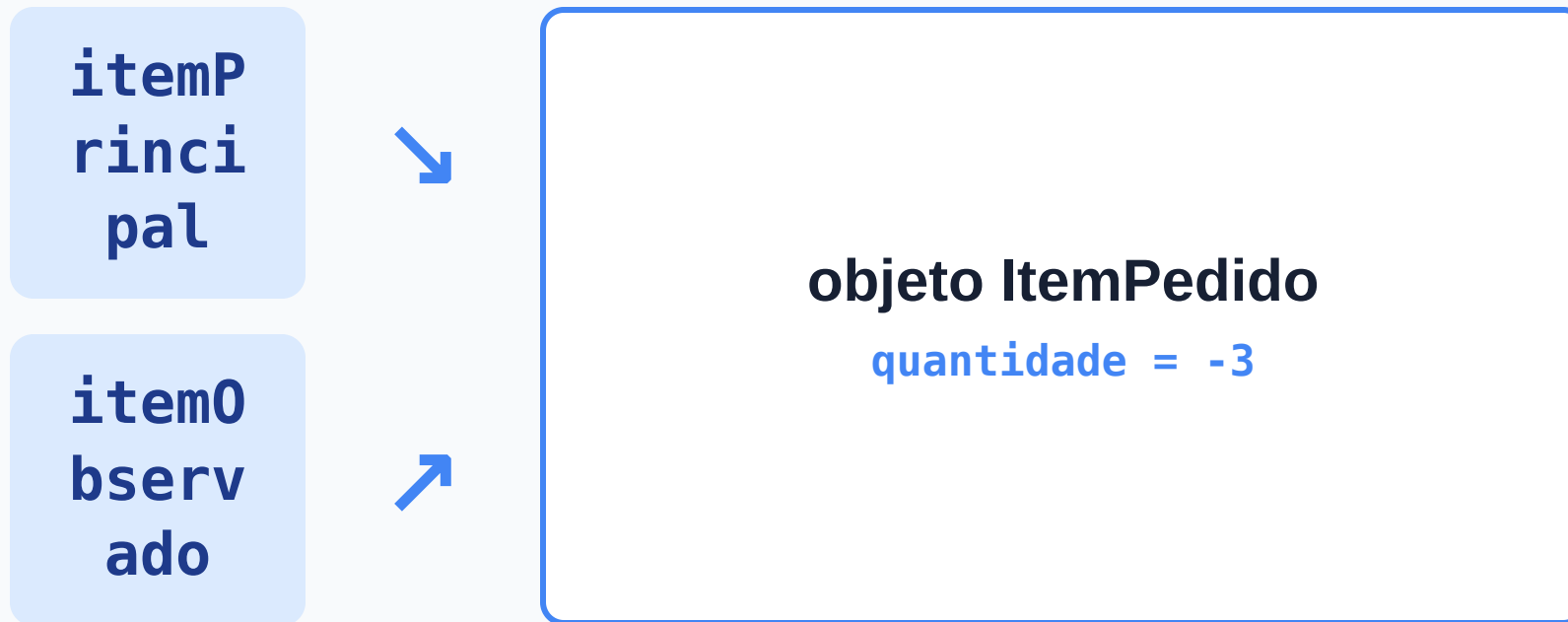
-3

Subtotal calculado

```
itemPrincipal.calcularSubtotal()
```

-450.0

A alteração chegou ao único objeto



A alteração foi feita por um nome e observada pelo outro.

O PROBLEMA

O Java aceita. O domínio não.

Linguagem

`-3` é um valor possível de `int`.
A atribuição é aceita.

Item de pedido

Quantidade negativa não representa um estado válido.

O PROBLEMA

Quem deve decidir?

Se `quantidade` pertence ao estado de `ItemPedido`, por que qualquer trecho externo pode escolher livremente seu valor?

Precisamos criar uma fronteira sem impedir toda mudança legítima.

O campo passa a ser privado

```
public class ItemPedido {  
    String descricao;  
    double precoUnitario;  
    private int quantidade;  
  
    public double calcularSubtotal() {  
        return precoUnitario * quantidade;  
    }  
}
```

Antes e depois de `private`

Antes

```
itemObservado.quantidade = -3;
```

acesso externo permitido

Depois

```
itemObservado.quantidade = -3;
```

acesso externo bloqueado

O erro é uma evidência

COMPILAÇÃO

```
quantidade has private access in ItemPedido
```

O código externo ainda conhece o objeto, mas perdeu o acesso direto ao campo.

private e public

- `private` restringe o acesso à própria classe;
- `public` disponibiliza uma classe ou operação para uso externo;
- métodos de `ItemPedido` continuam acessando `quantidade` ;
- código externo usa somente o que a classe disponibiliza.

E se não houver modificador?

```
private int quantidade; // própria classe  
    int quantidade; // mesmo pacote  
public int quantidade; // acesso externo
```

Sem modificador ocorre acesso de pacote (`package-private`).

ESCOPO DA MUDANÇA

Uma fronteira por vez

```
String descricao;  
double precoUnitario;  
private int quantidade;
```

Nesta etapa, apenas `quantidade` será protegida.

UMA NOVA NECESSIDADE

Como realizar uma mudança legítima?

Bloquear a atribuição direta impede o valor inválido.

Mas também impede que o pedido aumente de 5 para 7 .

Escolher o estado × solicitar uma operação

Atribuição direta

```
item.quantidade = 5;
```

código externo escolhe o estado

Comportamento

```
item.aumentarQuantidade(5);
```

código externo solicita uma operação

O objeto preserva a regra

```
public void aumentarQuantidade(int unidades) {  
    if (unidades > 0) {  
        quantidade += unidades;  
    }  
}
```

A mudança só acontece quando `unidades > 0`.

VERIFICANDO A REGRA

Três solicitações, dois resultados

```
ItemPedido item = new ItemPedido(); // quantidade começa em 0
item.aumentarQuantidade(2);          // quantidade passa a 2
item.aumentarQuantidade(-10);         // quantidade permanece 2
```

O valor inicial `0` é o padrão de um campo `int` sem inicialização explícita.

DICA

UM CONTRASTE IMPORTANTE

Campo não é variável local

```
class ItemPedido {  
    int quantidade; // campo: começa em 0  
}
```

```
void exemplo() {  
    int quantidade;  
    System.out.println(quantidade); // não compila  
}
```

OUTRA NECESSIDADE

Como observar o resultado?

Depois de proteger o campo, isto também deixa de compilar:

```
System.out.println(item.quantidade);
```

Consultar o estado não precisa devolver a liberdade de alterá-lo.

Uma operação para consultar

```
public int getQuantidade() {  
    return quantidade;  
}
```

```
System.out.println(item.getQuantidade());
```


CAPACIDADES DIFERENTES

Consultar não é alterar

Consulta

```
getQuantidade()
```

informa o estado atual

Mudança

```
aumentarQuantidade(...)
```

solicita uma alteração com regra

Se existe aumentar, precisa existir diminuir?

Não criamos operações porque elas parecem simétricas.

Uma operação pública representa uma capacidade necessária para o problema.

ATIVIDADE

CAPACIDADES DO OBJETO

Uma nova operação, novas decisões

Quantidade atual: **5**

- reduzir **2** → ?
- reduzir **10** → ?
- reduzir **-2** → ?

O problema realmente precisa oferecer essa operação?

VOLTANDO ÀS REFERÊNCIAS

O objeto continua compartilhado

```
itemObservado.aumentarQuantidade(2);  
System.out.println(itemPrincipal.getQuantidade());
```

Se a quantidade era 5, o que será exibido?

A fronteira não muda a identidade



As duas referências chegam ao mesmo objeto; o acesso externo acontece pelas operações disponíveis.

UMA SOLUÇÃO PLAUSÍVEL

E se criarmos um setter?

```
public void setQuantidade(int novaQuantidade) {  
    quantidade = novaQuantidade;  
}
```

O campo está privado. O estado está realmente protegido?

Setter × comportamento intencional

As duas soluções protegem o estado da mesma forma?

1. responda individualmente;
2. discuta a justificativa com um colega;
3. responda novamente;
4. prepare uma explicação para o fechamento coletivo.

Campo privado não garante encapsulamento

`setQuantidade(...)`

sem regra, o código externo continua escolhendo qualquer valor final

`aumentarQuantidade(...)`

expressa uma intenção e o objeto verifica a mudança solicitada

O contexto decide quais operações são apropriadas; setters não são sempre inadequados, mas o nome sozinho não garante proteção.

TRANSFERINDO O RACIOCÍNIO

Protegemos a quantidade. E o resto?

```
item.precoUnitario = -200.0;  
item.descricao = "";
```

O problema terminou quando protegemos quantidade ?

ATIVIDADE

TRANSFERINDO O RACIOCÍNIO

Que outros estados ainda exigem decisões?

1. preço negativo representa um estado válido?
2. descrição vazia representa um estado válido?
3. quem deveria decidir isso?

Não implemente a refatoração completa.

Encapsulamento

O objeto controla como seu estado pode ser consultado e alterado.

- campos privados ajudam a estabelecer a fronteira;
- operações públicas oferecem capacidades ao código externo;
- comportamentos podem preservar regras;
- encapsular não é gerar getters e setters automaticamente.

UMA QUESTÃO FUTURA

A alteração foi rejeitada. Quem chamou sabe disso?

```
item.aumentarQuantidade(-10);
```

Hoje garantimos que o estado permanece válido.

Como comunicar a rejeição é outra decisão de projeto.

Quem controla o estado?

- `private` estabelece uma fronteira contra o acesso direto;
- código externo solicita operações em vez de escolher livremente o estado;
- comportamentos podem preservar regras de mudança;
- consulta e alteração são capacidades diferentes;
- encapsulamento organiza esse controle de forma intencional.

E o estado inicial?

```
ItemPedido item = new ItemPedido();
```

Como garantir que um objeto já nasça com os dados necessários e em um estado válido?

Construtores entrarão nessa história depois. Ainda não precisamos estudá-los agora.